

Binance HFT справочная информация

Основные термины

Сетевые термины

WebSocket - протокол постоянной двусторонней связи поверх TCP между биржей и клиентом

TLS/SSL - протоколы шифрования соединения

TCP/IP - стек протоколов передачи данных

DNS Resolution - преобразование доменного имени в IP-адрес

Handshake - процесс установления соединения (TLS handshake)

Ping/Pong — механизм проверки того, работает ли сейчас WebSocket соединение (Binance проверяет на связи ли клиент (мы))

Архитектурные термины

RAII (Resource Acquisition Is Initialization) - идиома управления ресурсами в C++ (ресурс захватывается в конструкторе и автоматически освобождается в деструкторе, для RAII оберток часто используют умные указатели)

Callbacks - функции обратного вызова (в TLS контексте)

Buffer - массив для временного хранения данных

Frame - WebSocket фрейм (единица передачи данных)

Данные и форматирование

JSON (JavaScript Object Notation) - формат данных от Binance

Payload - полезная нагрузка фрейма (тиковые данные) [

```
{"e": "bookTicker", "u": 9436289333914, "s": "BTCUSDT", "b": "92196.10", "B": "2.878", "a": "92196.20", "A": "0.018", "T": 1765493529999, "E": 1765493529999}
```

]

Opcode - код операции в WebSocket фрейме (0x81, 0x89, 0x8A)

Timestamp - временная метка (в наносекундах от 1970 1 января)

Market Data - рыночные данные (котировки, стаканы)

Переменные

Symbol - торговая пара (например, BTCUSDT)

Stream - поток данных (например, bookTicker)

fstream.binance.com - endpoint для получения рыночных данных

SUBSCRIBE - метод подписки на данные

Технические составляющие

Botan - криптографическая библиотека

Boost.Log - библиотека логирования

Async I/O - асинхронный ввод/вывод

Socket Descriptor - дескриптор сокета

Global Buffer - глобальный буфер фреймов (*g_data*)

Обработка ошибок

Exception Handling - обработка исключений

Error Status Codes - коды статусов ошибок

Connection Health Check - проверка работы соединения

Оптимизации

Zero-copy - подход без копирования данных (константные ссылки, *string_view* и т.п.)

In-place Processing - обработка на месте без перемещения данных

Static Allocation - статическое выделение памяти (заранее выделили, много раз используем)

Архитектура проекта

1. Обзор системы

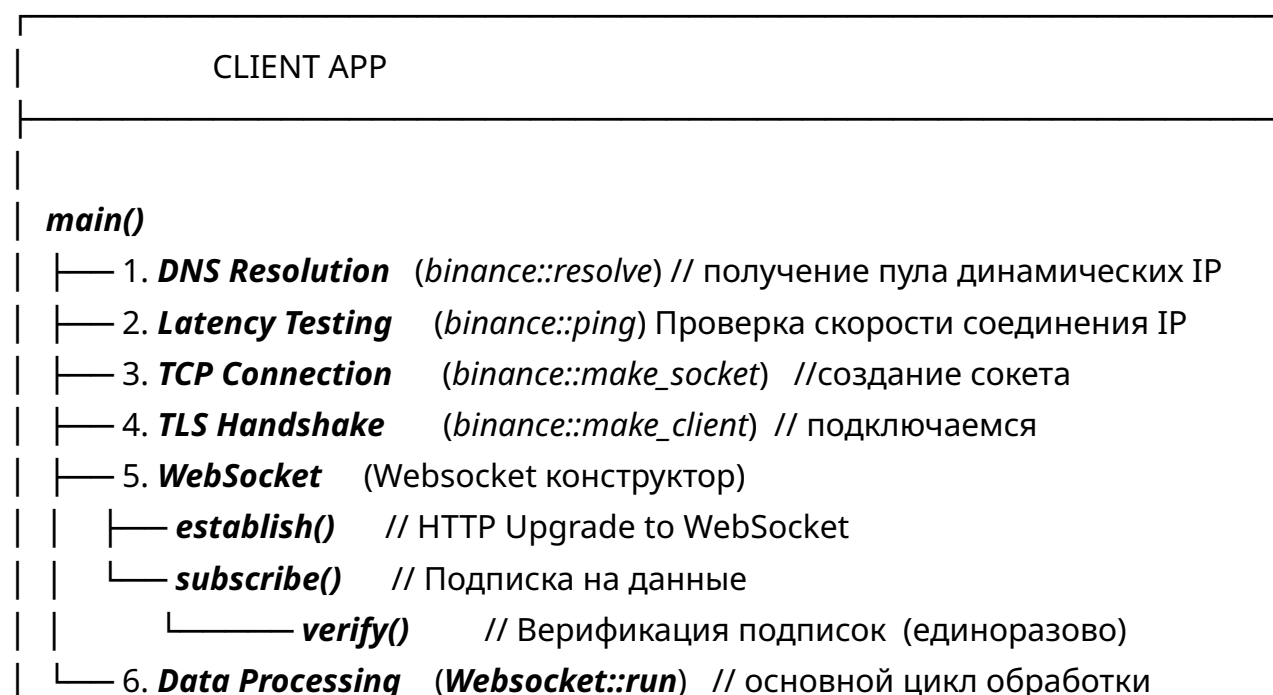
1.1. Назначение

На данный момент программа представляет собой клиент для сбора рыночных данных с Binance через WebSocket соединение. Цель - установить защищенное соединение, подписаться на торговые пары и собирать данные для дальнейшего анализа.

1.2. Ключевые характеристики

- **Защищенное соединение:** TLS с проверкой сертификатов подлинности
- **Логирование:** Подробное логирование операций
- **Обработка данных с биржи:** Парсинг WebSocket фреймов в реальном времени
- **Сохранение результатов:** Запись данных в файл после накопления 100к

2. Схема работы программы



```
|  └─ save()      // Сохранение данных в файл
```

3.1. Модуль сетевого взаимодействия

Назначение: Преобразование доменного имени в IPv4 адреса в список из 8 динамических IP — адресов.

1. Подготовка hints для getaddrinfo (AF_INET, SOCK_STREAM) (подсказки)
2. Вызов getaddrinfo() для "fstream.binance.com"
3. Итерация по результатам, преобразование sockaddr_in в строку
4. Возврат вектора строк с IP-адресами (8 штук)

1. Для каждого IP из resolve() Формирование команды: nmap <IP> -p 443
2. Запуск через Pipe (RAII-обертка для popen)
3. Парсинг вывода с regex "(\\([0-9.]+\\)s latency\\))" (регулярные выражения)
4. Логирование latency через LOGGER_PUT_DEBUG

- **Класс *binance::Socket*** : RAII-обертка для дескриптора сокета с автоматическим закрытием его в деструкторе

3.2. TLS модуль

3.2.1. TLS Client Initialization (binance::make_client)

Назначение: Установка TLS соединения через библиотеку Botan

Компоненты:

1. **Callbacks** (наследник Botan::TLS::Callbacks)
 - **tls_emit_data()**: Отправка зашифрованных данных в сокет через **send()**
 - **tls_record_received()**: Прием расшифрованных данных в глобальный буфер *g_data*
 - **recv(socket, buffer, size)** - записывает данные из сокета в буфер размера *size*.
 - **tls_alert()**: Логирование TLS алертов
 - **tls_session_established()**: Логирование установленной сессии и ciphersuite(набора шифров)
 - **tls_verify_cert_chain()**: Валидация цепочки сертификатов
2. **Policy** (наследник Botan::TLS::Policy)
 - **allowed_ciphers()**: Возвращает {"AES-128/GCM"} (алгоритм шифрования)
3. **Botan::TLS::Client** : Основной TLS клиент

3.2.2. TLS Handshake Process (make_client())

Назначение: Создание HTTP клиента и подключение к бирже.

Алгоритм работы:

1. Создание shared_ptr на AutoSeeded_RNG, Session_Manager_In_Memory, Credentials_Manager, Policy, Callbacks
2. Инициализация Botan::TLS::Client с Server_Information(host, port)
3. Цикл **while (!client->is_active())**: получение данных через **recv()** и передача в **client**→**received_data()** пока не подключились успешно.

3.3. WebSocket модуль

3.3.1. Класс binance::Websocket

Назначение: Управление WebSocket соединением и обработка данных, полученных через wss.

3.3.2.Инициализация (единоразовый вызов в конструкторе WebSocket)

Алгоритм работы:

1. establish():

HTTP Upgrade до WebSocket

- Отправка обновления через client->send():

~

GET /ws HTTP/1.1

Host:fstream.binance.com:443

Connection:Upgrade

Upgrade:websocket

Sec-WebSocket-Version:13

~

2. subscribe():

Назначение: Подписка на рыночные данные

Алгоритм работы:

- Для каждого символа в s_symbols ["btcusdt", ...]
- Для каждого стрима в s_streams ["bookTicker", ...]
- Формирование JSON: {"method":"SUBSCRIBE","params":["[btcusdt@bookTicker](#)", ..., "id":0}
- Упаковка в WebSocket фрейм: "\x81" + **char(size)** + payload
- Оправка сформированного пакета через **client->send()**

3. verify():

Назначение: Верификация ответов (проверка по размеру полученного сообщения) (единоразово)

Алгоритм работы:

- Ожидание накопления (1 подписка + 2) * 1024 байт в g_data
- Поиск 2 фреймов: 1 подтверждение подписки, 1 начало данных
- Проверка подтверждения на наличие "result":null
- Установка m_index на начало первого фрейма с данными

3.3.3. Основной цикл обработки (WebSocket::run())

Назначение: обработка входящих фреймов.

Алгоритм работы:

while (true) {

0. Инициализация переменных(размер фрейма, переменные для итерирования(i,j,k), Размер полезной нагрузки текущего фрейма, глобальный буфер для хранения фреймов)

1. Прием данных через **recv()** → **client->received_data()** → **g_data** (запись в глобальный буфер сразу при получении)

2. Парсинг WebSocket фреймов в глобальном буфере **g_data** (чтение полученных фреймов из буфера) итерация по ним.

3. Для каждого фрейма (начиная с **m_index**):

- Определение размера **payload** (первый байт [**m_index + 1**]):

* Если 126: размер в байтах с номерами 2-3 (то есть реальный размер полезной нагрузки содержится в следующих 2х байтах (16 битное число))
-тогда размер высчитывается как **size_t** от двух склеенных байтов (2 и 3)

* Если <126: это реальный размер

-тогда в **payload_size** пишем этот байт скастованный к **size_t**

- Если **i + payload > g_size** — значит нам пришел обрубок, он обрабатывается отдельно.

- Обработка по **opcode**:

- 0x81 (данные): сохранение **payload** в **m_frames** в паре с наносекундным timestamp

- 0x89 (**ping**): отправка **pong** (изменение 0x89 на 0x8a и отправка)

4. Накопление 100 000 фреймов

5. При достижении лимита: вызов **save()** и возврат **Status::success**

}

3.4. Логирование

3.4.1. Класс **binance::Logger**

Назначение: Логирование операций в файл через **Boost.Log**

Алгоритм работы:

- Уровни:

1. **Fatal**: Критические ошибки, завершение программы

2. **Error**: Ошибки выполнения операций

3. **Debug**: Информация для отладки (latency, TLS алерты)

4. **Trace**: Трассировка входа/выхода из функций

- Формат: **00000000 | 2026 January 09 19:10:34.297998 UTC | 0x00007688 |**

```
0x0000704c40787780 | trace | message
```

- Инициализация через ***std::once_flag***

- Макросы:

* **LOGGER(logger)** - создание экземпляра с автоматическим trace сообщением логера в конструкторе/деструкторе

* **LOGGER_PUT_DEBUG / TRACE / ERROR / FATAL** — категории сообщений (запись в одну из категорий например : **LOGGER_PUT_DEBUG(logger, "ping received");**)

3.5. Управление данными

3.5.1. Глобальные буферы

uint8_t g_data[1'048'576]; --- Буфер для данных после **TLS** расшифровки

size_t g_size = 0; --- Текущий размер данных в буфере

Используются в **Callbacks::tls_record_received()** и **Websocket::run()**

3.5.2. Структура хранения накопленных данных

Назначение: Вектор ***m_frames*** пар ***payload + timestamp*** накопленных в классе **Websocket**

Алгоритм работы:

...

```
std::vector<std::pair<std::string, unsigned long long int>> m_frames;
```

...

- ***string: payload*** (JSON строка) (Пример такой строки:

```
{"e":"bookTicker","u":9436289333914,"s":"BTCUSDT","b":"92196.10","B":"2.878","a":"92196.20","A":"0.018","T":1765493529999,"E":1765493529999}
```

)

- ***unsigned long long***: наносекундный **timestamp** от **high_resolution_clock** (от начала времен 1970 год 1 января)

4. Конфигурация и настройки

4.1. Статические настройки

В Websocket:

```
static inline array<string_view, 1> const s_symbols = {"btcusdt"};  
static inline array<string_view, 1> const s_streams = {"bookTicker"};
```

В main():

```
const string_view host = "fstream.binance.com";  
const uint16_t port = 443;  
const string_view path = "/ws";
```

4.2. Лимиты

```
const size_t MAX_FRAMES = 100'000;    // Лимит количества фреймов перед  
сохранением  
const size_t BUFFER_SIZE = 1'048'576; // Размер g_data (основного буфера  
фреймов)(1MB)
```

5. Обработка ошибок

5.1. Уровни ошибок в Logger

1. **Fatal**: Критические ошибки, завершение программы
2. **Error**: Ошибки выполнения операций

5.2. Механизмы обработки ошибок

Все ошибки обрабатываются через *исключения*:

- В main() try-catch блок ловит *std::exception* и логирует как FATAL
- **WebSocket::run()** возвращает объект типа **Status enum** при завершении